

Machine Learning · Week 9

Reinforcement Learning

Markov Decision Process · Q-Learning · CNN & DQN · Pong Game Case Study

D4 Robotics Engineering Study Program

Department of Electrical Engineering · Politeknik Negeri Batam



Agenda Perkuliahan

Topik yang akan dibahas hari ini

01

Reinforcement Learning

Konsep dasar, cara kerja & komponen utama

02

Markov Decision Process (MDP)

State, Action, Reward, Policy, Optimal Policy

03

Exploration vs Exploitation

Epsilon-Greedy Strategy

04

Q-Learning

Q-Table, Bellman Equation, Update Rule, Kode

05

Case Study: Pong Game

Observation, Frame Stacking, CNN, DQN, Training

06

Deep Reinforcement Learning

Mengapa Neural Network diperlukan dalam RL

Apa itu Reinforcement Learning?

Paradigma belajar melalui interaksi dan umpan balik



Definisi

Reinforcement Learning (RL) adalah paradigma machine learning di mana sebuah agent belajar mengambil keputusan optimal melalui interaksi langsung dengan environment-nya.

Agent tidak diajarkan secara eksplisit apa yang harus dilakukan. Ia belajar sendiri melalui trial and error, dipandu oleh sinyal reward yang diterima dari environment.

Tipe ML	Input Data	Sinyal Belajar	Contoh
Supervised	Data berlabel	Label/Target	Klasifikasi gambar
Unsupervised	Data tak berlabel	Pola internal	Clustering
Reinforcement	Interaksi env.	Reward/Punishment	Game, Robot



Sequential Decisions

Mengambil serangkaian keputusan secara berurutan dari waktu ke waktu



Goal-Oriented

Tujuan: memaksimalkan total cumulative reward jangka panjang



Trial & Error

Belajar dari percobaan nyata, bukan dari contoh berlabel



Delayed Reward

Reward bisa datang jauh setelah aksi diambil (sparse reward)

RL Cycle: Agent–Environment Interaction

Siklus interaksi berkelanjutan: Agent mengamati State, mengambil Action, dan menerima Reward dari Environment



02

Markov Decision Process

Fondasi matematis dari Reinforcement Learning



Markov Decision Process (MDP)

Kerangka formal untuk mendefinisikan masalah Reinforcement Learning

MDP adalah model matematika yang mendefinisikan masalah RL secara formal. Sebuah MDP terdiri dari 5 komponen utama: (S, A, R, P, γ). Asumsi utama MDP: keputusan masa depan hanya bergantung pada state saat ini (Markov Property).



State Space

Kumpulan semua kemungkinan kondisi lingkungan yang dapat diamati agent



Action Space

Kumpulan semua tindakan yang dapat dilakukan agent dalam setiap state



Reward Function

Fungsi yang memberikan sinyal numerik setelah agent mengambil aksi di state tertentu



Transition Prob

$P(s'|s,a)$: probabilitas berpindah ke state s' dari state s setelah aksi a



Discount Factor

Faktor diskon ($0 \leq \gamma \leq 1$): seberapa penting reward masa depan dibanding reward sekarang

>> Markov Property: $P(S_{t+1} | S_t, A_t) = P(S_{t+1} | S_0, A_0, \dots, S_t, A_t) \rightarrow$ State saat ini sudah cukup mewakili seluruh history

MDP: State (S) & Action (A)

Representasi kondisi lingkungan dan ruang tindakan agent

S

State Space (S)

Definisi:

State $s \in S$ adalah representasi kondisi lingkungan pada suatu titik waktu tertentu yang dapat diamati oleh agent.

Notasi Matematis:

$S_t \in S \rightarrow$ state pada waktu t

Contoh Nyata:

- >> Robot: koordinat posisi (x, y), orientasi, sensor jarak
- >> Chess: posisi seluruh bidak di papan catur (8x8 grid)
- >> Pong: posisi bola, posisi paddle, kecepatan bola

A

Action Space (A)

Definisi:

Action $a \in A$ adalah tindakan yang dipilih agent pada suatu state. Bisa berupa himpunan diskret atau kontinu.

Notasi Matematis:

$A_t \in A(S_t) \rightarrow$ aksi pada waktu t

Contoh Nyata:

- >> Robot: maju, mundur, belok kiri, belok kanan
- >> Chess: setiap kemungkinan langkah bidak yang legal
- >> Pong: naik, turun, diam (3 aksi diskret)

MDP: Reward Function $R(s, a)$

Sinyal numerik yang memandu proses pembelajaran agent

Reward Function $R(s, a)$ mendefinisikan seberapa baik atau buruk suatu aksi yang diambil dari sebuah state. Reward adalah satu-satunya sinyal yang mengarahkan agent untuk belajar perilaku yang diinginkan.

$$R(s, a) \rightarrow \mathbb{R}$$

|

$$R_{t+1} = R(S_t, A_t)$$

Penjelasan Variabel:

s State saat ini dari environment yang diamati agent

a Action yang dipilih agent dari state s

$R(\cdot)$ Nilai numerik reward yang diterima (bisa positif, negatif, atau nol)

R_{t+1} Reward yang diterima pada langkah t+1 setelah mengambil aksi a_t dari state s_t

Contoh: Robot Pembersih Lantai

Maju ke area kotor	+10	Bersihkan area
Bergerak acak	0	Tidak berguna
Menabrak dinding	-10	Berbahaya
Baterai habis	-50	Gagal total

Tujuan RL: Memaksimalkan Expected Cumulative Reward $\rightarrow G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum \gamma^k R_{t+k+1}$

MDP: Policy (π) & Optimal Policy (π^*)

Strategi pengambilan keputusan agent dan target optimalnya

π

Policy $\pi(a | s)$

Policy adalah strategi atau aturan yang digunakan agent untuk memilih aksi dari sebuah state. Policy bisa bersifat deterministik atau stokastik.

Deterministik: $\pi(s) \rightarrow a$

Stokastik: $\pi(a|s) \rightarrow P(a|s)$

π^*

Optimal Policy $\pi^*(s)$

Optimal Policy π^* adalah policy yang memaksimalkan expected cumulative reward dari setiap state yang mungkin.

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Value Functions – Mengukur Kualitas State & Policy

State-Value Function $V^\pi(s)$

$$V^\pi(s) = E_\pi [G_t \mid S_t = s] = E_\pi [\sum \gamma^k R_{t+k+1} \mid S_t = s]$$

Expected return ketika berada di state s dan mengikuti policy π

Action-Value Function $Q^\pi(s,a)$

$$Q^\pi(s,a) = E_\pi [G_t \mid S_t=s, A_t=a] = E_\pi [\sum \gamma^k R_{t+k+1} \mid S_t=s, A_t=a]$$

Expected return ketika berada di state s , mengambil aksi a , lalu mengikuti policy π

Contoh MDP: Grid World (Robot Pembersih)

Ilustrasi komponen MDP dalam skenario nyata sederhana



Identifikasi Komponen MDP:



State Space

16 kemungkinan posisi (4x4 grid) = $\{(r,c) \mid 0 \leq r,c \leq 3\}$



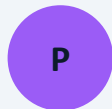
Action Space

4 aksi: {Atas, Bawah, Kiri, Kanan}



Reward Function

Goal = +100 | Obstacle = -10 | Langkah = -1



Transition

Deterministik (aksi selalu berhasil, kecuali dinding)



Discount Factor

$\gamma = 0.9$ (reward masa depan lebih rendah nilainya)

03

Q-Learning

Algoritma pencarian optimal policy berbasis nilai Q



Exploration vs Exploitation

Dilema fundamental dalam proses pembelajaran agent RL



Exploration

Mencoba aksi baru yang belum pernah atau jarang dilakukan, meskipun hasilnya belum tentu bagus. Tujuan: menemukan strategi baru yang mungkin lebih optimal.

Analogi: Mencoba restoran baru yang belum pernah dikunjungi — mungkin lezat atau kecewa



Exploitation

Menggunakan pengetahuan yang sudah ada untuk memilih aksi terbaik yang diketahui. Tujuan: memaksimalkan reward berdasarkan pengalaman yang telah terkumpul.

Analogi: Selalu makan di restoran favorit yang sudah terbukti lezat dan aman

Solusi: Epsilon-Greedy Strategy (ϵ -greedy)

Dengan probabilitas ϵ

Agent memilih aksi ACAK (exploration)
Menghindari terjebak pada solusi sub-optimal

Dengan probabilitas $(1 - \epsilon)$

Agent memilih aksi TERBAIK sejauh ini (exploitation)
Berdasarkan Q-value tertinggi untuk state saat ini

Contoh: $\epsilon = 0.1$

10% waktu $\rightarrow$ eksplorasi aksi acak
90% waktu $\rightarrow$ eksploitasi aksi terbaik

Q-Learning & Q-Table

Algoritma value-based yang menyimpan kualitas setiap pasangan (state, action)

Apa itu Q-Learning?

Q-Learning adalah algoritma RL yang belajar fungsi nilai $Q(s,a)$, yaitu seberapa bagus aksi a jika diambil dari state s dan mengikuti optimal policy setelahnya. Q-Learning tidak memerlukan model transisi environment (model-free).

Q-Table

Q-Table adalah matriks 2D (State $\times$ Action) yang menyimpan nilai Q untuk setiap pasangan. Setiap cell $Q(s,a)$ diperbarui selama training menggunakan Bellman Equation.

Contoh Q-Table (Robot Pembersih):

State / Action	Maju ↑	Mundur ↓	Kiri ←	Kanan →
(0,0) - Start	0.45	0.10	-0.20	0.62
(0,1) - Clear	0.78	0.32	0.55	0.41
(1,0) - Dirty	0.92	0.15	0.38	0.70
(3,3) - Goal	0.00	0.00	0.00	0.00

✓ Nilai Q tertinggi per baris (hijau) = Best Action yang dipilih agent di state tersebut

Bellman Equation & Q-Learning Update Rule

Fondasi matematika yang memperbarui nilai Q dari pengalaman

Bellman Optimality Equation untuk Q^* :

$$Q^*(s, a) = E [r + \gamma \cdot \max_{a'} Q^*(s', a') \mid s, a]$$

Q-Learning Update Rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

α (alpha)

Learning Rate ($0 < \alpha \leq 1$): kecepatan agent memperbarui nilai Q. Terlalu besar $\rightarrow$ tidak stabil; terlalu kecil $\rightarrow$ lambat belajar

γ (gamma)

Discount Factor ($0 \leq \gamma \leq 1$): seberapa penting reward masa depan. $\gamma=0 \rightarrow$ hanya peduli reward sekarang; $\gamma=1 \rightarrow$ peduli semua reward

r

Reward yang diterima setelah mengambil aksi a dari state s dan berpindah ke state s'

$\max_{a'} Q(s', a')$

Q-value maksimum dari state berikutnya s' . Mewakili prediksi terbaik reward masa depan (TD Target)

$[] = \text{TD Error}$

Temporal Difference Error: selisih antara target baru $[r + \gamma \cdot \max Q(s', a')]$ dan estimasi sebelumnya $Q(s, a)$

Contoh Update Q-Value (Step-by-Step)

Simulasi satu langkah pembelajaran Q-Learning dengan angka konkret

Diketahui:

State saat ini (s)	= (1, 0) – posisi robot di baris 1, kolom 0	Action dipilih (a)	= Maju ↑
Reward diterima (r)	= +10 (ditemukan area kotor)	State berikutnya (s')	= (0, 0) – robot berhasil maju
Q(s, a) sebelumnya	= 0.50	max Q(s', a')	= 0.80 (Q-value terbaik dari state baru)
Learning rate α	= 0.1	Discount factor γ	= 0.9

Langkah Perhitungan:

Langkah 1: Hitung TD Target

$$\text{TD Target} = r + \gamma \cdot \max Q(s', a') = 10 + 0.9 \times 0.80 = 10.72$$

Langkah 2: Hitung TD Error

$$\text{TD Error} = \text{TD Target} - Q(s, a) = 10.72 - 0.50 = 10.22$$

Langkah 3: Update Q(s, a)

$$Q(s, a)_{\text{baru}} = 0.50 + 0.1 \times 10.22 = 0.50 + 1.022 = 1.522$$

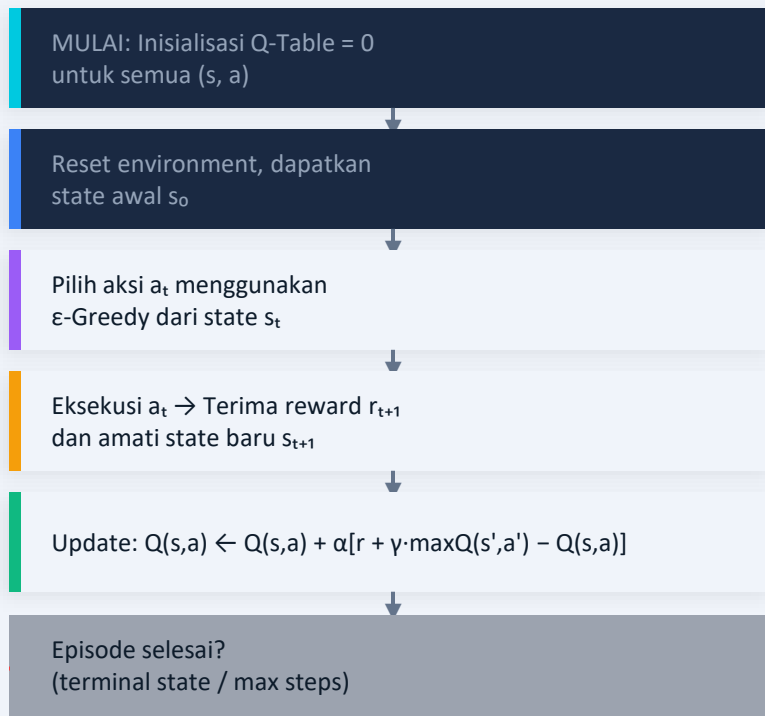
>> Hasil: Q((1,0), Maju) diperbarui dari 0.50 → 1.52 | Agent semakin yakin bahwa 'Maju' adalah aksi baik di state (1,0)

Alur Algoritma Q-Learning

Flowchart lengkap proses training agent menggunakan Q-Learning

TIDAK →
 $s_t \leftarrow s_{t+1}$, lanjutkan loop

>> Loop berlanjut
hingga Q-Table konvergen



YA →
Selesai / Episode baru

>> Konvergen: Q-values stabil
dan policy menjadi optimal

Implementasi Q-Learning: Kode Harrison

Program Q-Learning GUI — Robot belajar menuju Goal sambil menghindari rintangan (Tkinter)

1. Inisialisasi Parameter

```
ALPHA = 0.1      # Learning rate
GAMMA = 0.9      # Discount factor
EPSILON = 0.1    # Eksplorasi (10%)
EPISODES = 1000  # Jumlah episode training
```

2. Q-Table (State × Action)

```
# state: (row, col) | actions: 4 arah
Q = np.zeros((GRID_H, GRID_W, 4))
# shape: (6, 6, 4) untuk grid 6x6
```

3. Pilih Aksi (ϵ -Greedy)

```
if random.random() < EPSILON:
    action = random.randint(0, 3) # Eksplorasi
else:
    action = np.argmax(Q[s])      # Eksploitasi
```

4. Update Q-Value

```
td_target = r + GAMMA * np.max(Q[s_new])
td_error = td_target - Q[s][action]
Q[s][action] += ALPHA * td_error
```

>> Reward: +100 (Goal), -10 (Rintangan), -1 (Langkah biasa) | Setelah training, agent pilih max Q(s) di setiap state untuk jalur optimal

04

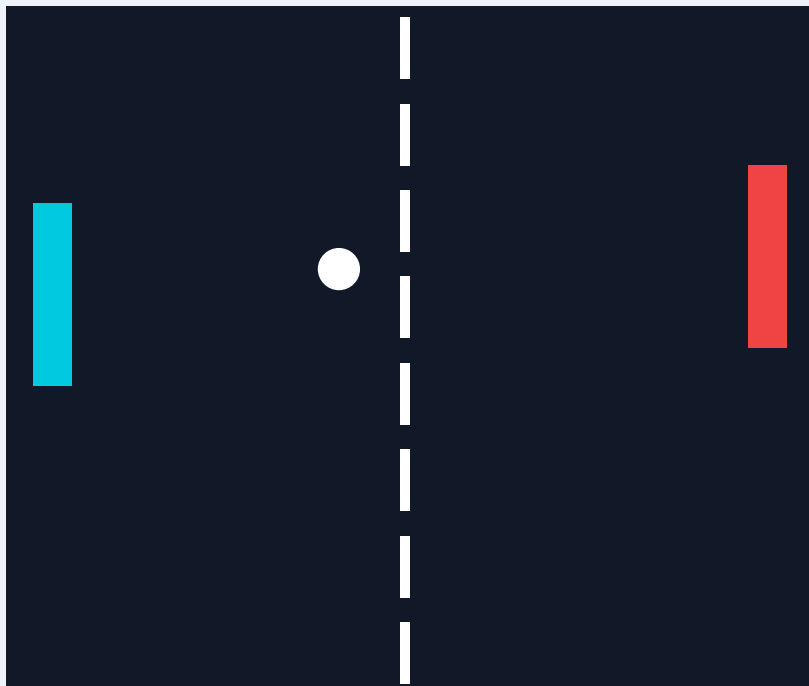
Case Study: Pong Game

Menerapkan RL pada permainan klasik Atari Pong



Cara Kerja Pong & Peran RL

Permainan klasik Atari sebagai testbed Reinforcement Learning



AGENT

Ball

OPP.



Gameplay Dasar

Dua pemain mengontrol paddle untuk memantulkan bola. Poin dicetak ketika bola melewati paddle lawan. Agent mengontrol paddle kiri sebagai 'player 1'.



Peran Agent dalam RL

Agent mengamati frame game sebagai state, memilih aksi (naik/turun/diam), dan menerima reward berdasarkan hasil aksi tersebut.



Proses Training

Agent bermain ribuan episode, belajar dari reward/punishment hingga mampu mengalahkan lawan dengan strategi optimal.

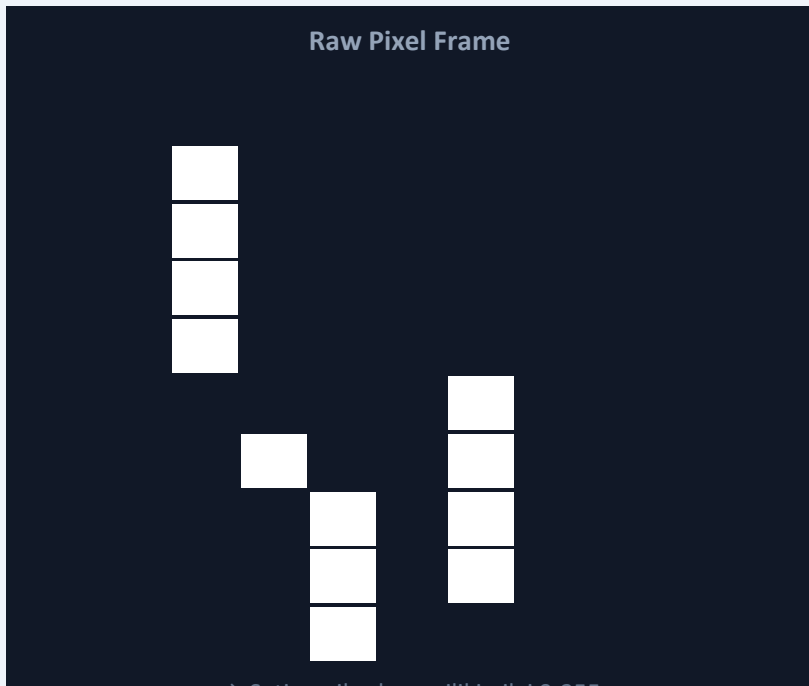


Komponen RL dalam Pong

State=frame piksel, Action={naik,turun,diam}, Reward=+1 (cetak poin) / -1 (kecolongan) / 0 (rally berlanjut).

Bagaimana Agent 'Melihat' Pong?

Agent tidak melihat seperti manusia — ia menerima data piksel mentah



→ Setiap piksel memiliki nilai 0-255

Apa itu Observation?

Observation adalah representasi state yang diterima agent dari environment. Pada Pong, observation berupa array piksel dari layar game: shape (210, 160, 3) untuk RGB, atau (84, 84, 1) setelah preprocessing.

Preprocessing Frame

Frame asli (210×160×3 RGB) diubah melalui: grayscale → resize ke 84×84 → normalize nilai 0-1. Ini mengurangi dimensi input secara signifikan dan mempercepat training.

Frame → Tensor

Setelah preprocessing, frame diubah menjadi tensor (array multidimensi) berbentuk [84, 84, 1]. Nilai float mewakili intensitas setiap piksel untuk diproses oleh neural network.

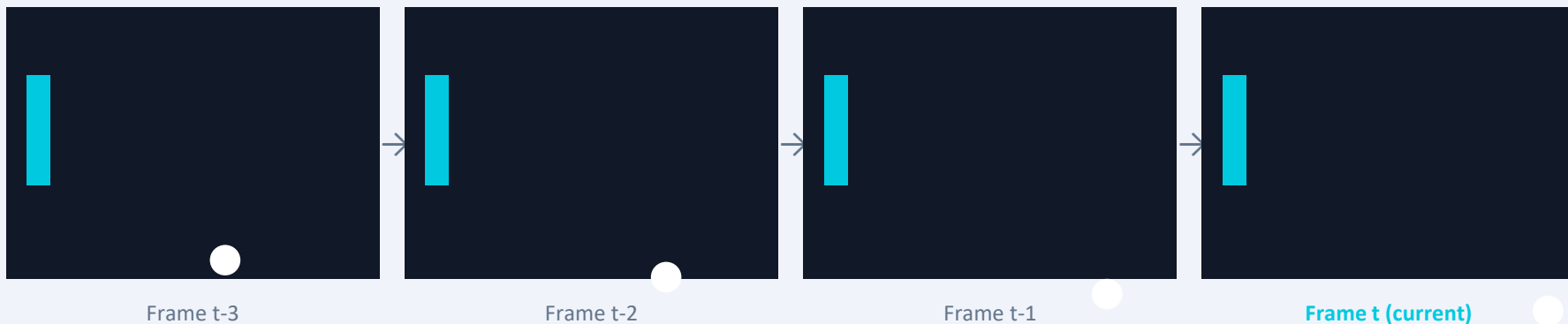
>> Shape tensor setelah preprocessing: (84, 84, 1) → nilai float 0.0–1.0 | 210×160×3 (raw) → 84×84×1 (processed) → ±85% pengurangan dimensi!

Frame Stacking: Memahami Gerakan Bola

Satu frame tidak cukup — agent butuh urutan frame untuk menentukan arah & kecepatan

Masalah: Satu Frame = Informasi Tidak Lengkap

Satu frame hanya menunjukkan POSISI bola, bukan ARAH atau KECEPATAN gerakan. Agent tidak bisa memprediksi ke mana bola akan pergi hanya dari satu snapshot.



Solusi: Frame Stacking

Stack 4 Frame

4 frame terakhir digabungkan menjadi satu input → shape: (84, 84, 4)

Deteksi Gerak

Perubahan posisi bola antar frame = agent mengetahui arah & kecepatan

Temporal Context

Agent mendapat informasi dinamis (bola bergerak ke mana) bukan hanya snapshot statis

Sistem Reward & Punishment pada Pong

Sinyal numerik yang membimbing agent belajar strategi optimal

Definisi Reward Function $R(s, a)$ pada Pong:

Kejadian / Event	Reward	Tipe	Penjelasan
Agent mencetak poin (bola melewati lawan)	+1	POSITIF	Mendorong agent untuk menyerang
Berhasil memantulkan bola (rally berlanjut)	0	NETRAL	Setiap langkah tanpa akhir tidak diberi nilai
Lawan mencetak poin (bola melewati agent)	-1	NEGATIF	Mendorong agent untuk bertahan lebih baik

Alur Interaksi Agent–Environment pada Pong:



Tujuan Agent: Memaksimalkan total cumulative reward sepanjang game → Menang sebanyak mungkin set!
Formula: $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$ Agent belajar memantulkan bola BUKAN karena diperintah, melainkan karena reward +1 lebih besar dari -1.

CNN: Membaca Frame Pong dengan Deep Learning

Convolutional Neural Network memproses gambar game secara otomatis



Mengapa Perlu CNN?

Q-Learning tradisional memerlukan tabel Q berukuran raksasa untuk semua piksel. CNN mengekstrak fitur visual secara otomatis dari gambar tanpa perlu mendefinisikan fitur secara manual.

Arsitektur CNN untuk Pong (DQN):



Yang CNN Pelajari Secara Otomatis:

Tepi & Garis: Conv layer awal mendeteksi tepi paddle dan bola

Bentuk & Posisi: Conv layer tengah mengenali posisi objek

Pola Kompleks: FC layer mengintegrasikan fitur untuk nilai Q

Deep Q-Network (DQN)

Menggabungkan Q-Learning dengan Deep Neural Network untuk state space besar

Q-Learning vs Deep Q-Network:

Aspek	Q-Learning	Deep Q-Network (DQN)
Representasi Q	Q-Table (matriks 2D)	Neural Network (CNN + FC)
State Space	Terbatas (jumlah baris harus terhitung)	Tak terbatas (raw pixels, kontinu)
Generalisasi	Tidak bisa generalisasi antar state	Bisa generalisasi pola visual mirip
Skalabilitas	Tidak skala untuk input berdimensi tinggi	Skala ke Atari 49 games, robotik

Inovasi Kunci DQN (DeepMind, 2013):

Experience Replay

Simpan (s, a, r, s') dalam replay buffer. Sample acak untuk melatih network → memecah korelasi antar pengalaman berurutan

Target Network

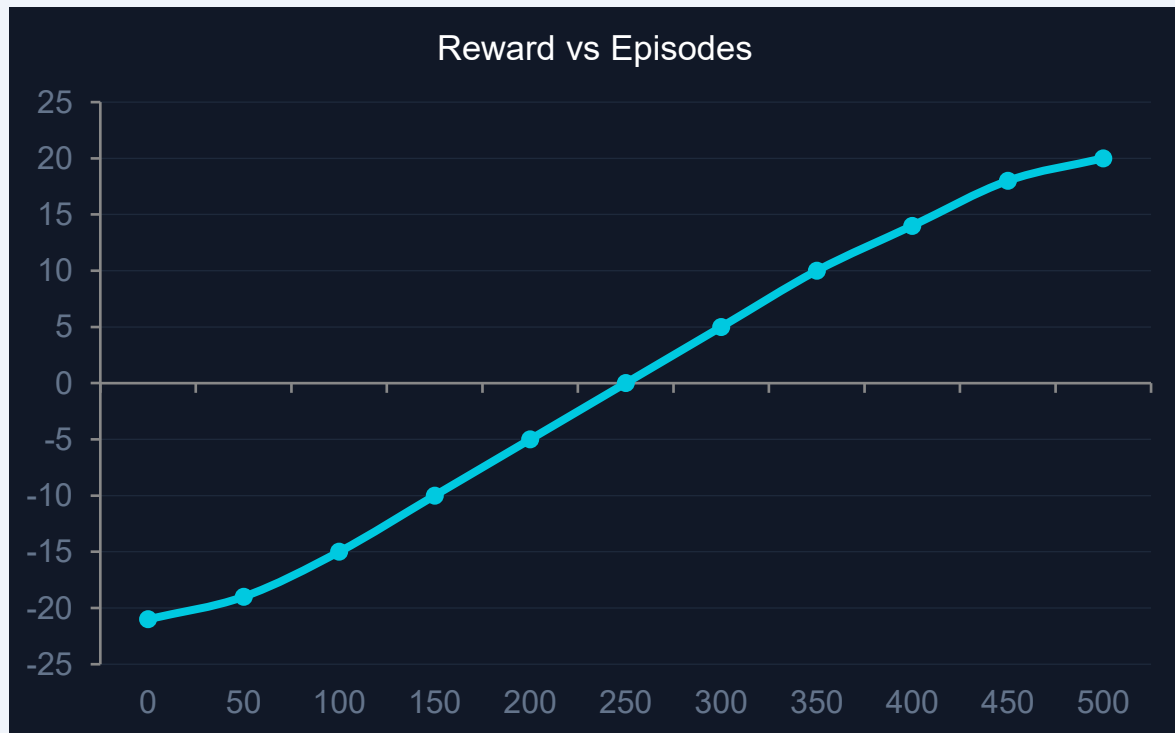
Gunakan 2 jaringan: online network (dilatih) dan target network (periodik update). Menstabilkan training yang rentan divergen

CNN Input

Menerima 4 stacked frames 84×84 grayscale sebagai input. CNN secara otomatis mengekstrak fitur visual tanpa feature engineering manual

Progres Training DQN pada Pong

Agent belajar dari random play hingga mengalahkan lawan secara konsisten



Milestone Pembelajaran:

Ep. 0–100: Bermain Acak

Agent tidak tahu apa yang dilakukan. Reward sangat buruk (≈ -21)

Ep. 100–200: Mulai Belajar

Agent mulai belajar cara memantulkan bola sederhana

Ep. 200–350: Memperbaiki

Policy mulai konsisten. Reward meningkat pesat menuju 0

Ep. 350–500: Mahir

Agent mampu mengalahkan built-in AI secara konsisten (reward > 15)

>> Training DQN Pong membutuhkan ~10 juta frames (± 500 episode) pada GPU. Tanpa GPU, bisa memakan waktu berjam-jam hingga berhari-hari!

05

Deep Reinforcement Learning



Ketika RL bertemu Deep Learning untuk masalah kompleks

Deep Reinforcement Learning

Kombinasi Deep Learning dan RL untuk menyelesaikan masalah berdimensi tinggi

RL Tradisional (Q-Table) tidak mampu menangani state space yang sangat besar seperti piksel game atau sensor robot nyata. Deep RL menggunakan Neural Network sebagai pengganti Q-Table untuk mengestimasi Q-value secara langsung dari input mentah.

RL Tradisional

- X Q-Table: rows = states, cols = actions
- X Hanya untuk state space kecil & diskret
- X Tidak bisa menangani input piksel/gambar
- X Tidak ada generalisasi antar state mirip
- X Cepat konvergen pada masalah kecil

Deep RL (DQN & Beyond)

- ✓ Neural Network menggantikan Q-Table
- ✓ Dapat menangani state space kontinu & besar
- ✓ Menerima raw pixels atau sensor data langsung
- ✓ Generalisasi fitur visual secara otomatis (CNN)
- ✓ Butuh lebih banyak data & komputasi (GPU)

Aplikasi Deep RL di Dunia Nyata:

Atari Games: 49 game score
superhuman

AlphaGo: Kalahkan juara dunia Go

Robotika: Manipulator, locomotion

Self-Driving: Perencanaan jalur
otonomi

Drug Discovery: Optimasi molekul
protein

Referensi & Sumber Belajar

[1]

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. [Buku teks utama RL]

[2]

Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533. [Paper DQN original DeepMind]

[3]

GeeksForGeeks. Q-Learning in Python. <https://www.geeksforgeeks.org/q-learning-in-python/>

[4]

Dewa, C. K. (n.d.). Pengantar Dasar Deep Reinforcement Learning. Jurusan Informatika, Universitas Islam Indonesia.

[5]

OpenAI. Spinning Up in Deep RL. <https://spinningup.openai.com/> [Panduan lengkap Deep RL dengan kode]

[6]

Sentdex / Harrison (YouTube). Reinforcement Learning Q-Learning with Python. [Tutorial kode Q-Learning GUI]

Terima Kasih!

Any Questions?

Week 9 — Reinforcement Learning

Machine Learning · D4 Robotics Engineering · Politeknik Negeri Batam

